Trapezoidal Rule

The **Trapezoidal Rule** is a fundamental method in numerical integration used to approximate the value of a definite integral of the form ${}^{b}\int_{a} f(x)\, dx$. It estimates the area under the curve $y = f(x)$ by dividing the interval [a, b] into smaller subintervals and approximating the region under the curve as a series of trapezoids.

Formula For Trapezoidal Rule

Let $y = f(x)$ be a continuous curve defined on the closed interval [a, b]. Now we divide the closed interval [a, b] into n equal subintervals, with each having the width of, $h = \Delta x = (b - a)/n$

Such that, $a = x_0 < x_1 < x_2 < \cdots < x_n = b$

Now, using the Trapezoidal rule formula, we can find the area under the curve as,
$\int^{b}_{a} f(x)\, dx$ = Area Under the Curve = $(\Delta x/2)\, [y_0 + 2\,(y_1 + y_2 + y_3 + \ldots\ldots + y_{n-1}) + y_n]$

where, $y_0, y_1, y_2, \ldots\ldots y_n$ are the values of function at x = 1, 2, 3, ..., n respectively.

Solved Example of Trapezoidal Rule

Example 1: Find the area under the curve using the Trapezoidal Rule for the following points:

x	0	1	2	3
y	2	4	8	16

Solution:

The width of the interval is $h = x_{i+1} - x_i = 1$

Apply the Trapezoidal Rule Formula: $\int_a^b f(x)dx \approx \frac{h}{2}[y_0 + 2\Sigma_{i-1}^{n-1}(y_i) + y_n]$

Here:

- $y_0 = 2, y_1 = 4, y_2 = 8, y_3 = 16$
- $n = 3$

${}^{3}\int_{0} f(x)\, dx \approx ½[2 + 2(4 + 8) + 16]$
$= ½[2 + 24 + 16]$
$= ½[42]$
$= 21$

Example 3: Find the area enclosed by the function f(x) between x = 0 to x = 4 with 4 intervals: f(x) = 4x

Solution:

Here a = 0, b = 4 and n = 4.

$$\Delta x = \frac{b-a}{n}$$
$$= \Delta x = \frac{4-0}{4}$$
$$= \Delta x = 1$$

$$x_i = a + i\Delta x$$
$$= x_i = 0 + i$$
$$= x_i = i$$

The trapezoidal rule for n = 4 is,

$$T_n = \frac{\Delta x}{2}(f(x_0) + 2f(x_1) + 2f(x_2) + 2f(x_3) + f(x_4))$$

Substituting the values in this equation,

$$T_n = \frac{\Delta x}{2}(f(x_0) + 2f(x_1) + 2f(x_2) + 2f(x_3) + f(x_4))$$

T_n = ½[0 + 2(4 + 8 + 12) + 16]
= ½[0 + 48 + 16]
= ½ [64]
= 32

Matrix normalization

Data normalization is a preprocessing method that resizes the range of feature values to a specific scale, usually between 0 and 1. It is a feature scaling technique used to transform data into a standard range. Normalization ensures that features with different scales or units contribute equally to the model and improves the performance of many machine learning algorithms.

Key Features of Normalization:

- Maps the minimum and maximum of a feature to a defined range
- Preserves the relative relationships of the original data
- Useful for algorithms that rely on distance metrics such as k-Nearest Neighbours and clustering

Difference Between Normalization and Standardization

Standardization, also called Z-score normalization is a separate technique. It transforms data so that it has a mean of 0 and a standard deviation of 1.

Key Features of Standardization:

- Centers the data around zero
- Scales according to the variability (standard deviation)

- Useful for algorithms that assume Gaussian distributions such as linear regression,logistic regression and neural networks.
- Less sensitive to outliers compared to Min-Max normalization.

There are several techniques to normalize data, each transforming values to a common scale in different ways

1. Min-Max Normalization

Min-Max normalization rescales a feature to a specific range, typically [0, 1]:

$$X_{\text{normalized}} = \frac{X - X_{\text{min}}}{X_{\text{max}} - X_{\text{min}}}$$

- The minimum value maps to 0
- The maximum value maps to 1
- Other values are scaled proportionally

2. Decimal Scaling

Decimal scaling normalizes data by shifting the decimal point of values:

$$v' = \frac{v}{10^j}$$

j is the smallest integer such that the maximum absolute value of v′*v*′ is less than 1

3. Logarithmic Transformation

Log transformation compresses large values and spreads out small values:

$$X' = \log(X + 1)$$

- Reduces skewness in data
- Stabilizes variance across features

4. Unit Vector (Vector) Normalization

Scales a data vector to have a magnitude of 1:

$$X' = \frac{X}{||X||}$$

- Commonly used in text mining and machine learning algorithms like KNN
- Preserves direction but normalizes magnitude

Implementation in Python

We will demonstrate how to normalize and standardize features in Python using scikit-learn library.

1. Import Required Libraries

We will import the necessary libraries like pandas and scikit-learn.

```
import pandas as pd
from sklearn.preprocessing import MinMaxScaler, StandardScaler
```

2. Loading the Dataset

We will load the dataset and separate the features from the target variable.

```
df = pd.read_csv('/content/heart.csv')
X = df.drop('target', axis=1)
y = df['target']
df.head()
```

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	ca	thal	target
0	63	1	3	145	233	1	0	150	0	2.3	0	0	1	1
1	37	1	2	130	250	0	1	187	0	3.5	0	0	2	1
2	41	0	1	130	204	0	0	172	0	1.4	2	0	2	1
3	56	1	1	120	236	0	1	178	0	0.8	2	0	2	1
4	57	0	0	120	354	0	1	163	1	0.6	2	0	2	1

3. Normalising the Features

We will normalize selected numeric features to scale them between 0 and 1.

```
features = ['age','trestbps','chol','thalach','oldpeak']
scaler = MinMaxScaler()

X_normalized = X.copy()
X_normalized[features] = scaler.fit_transform(X[features])
X_normalized.head()
```

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	ca	thal
0	0.708333	1	3	0.481132	0.244292	1	0	0.603053	0	0.370968	0	0	1
1	0.166667	1	2	0.339623	0.283105	0	1	0.885496	0	0.564516	0	0	2
2	0.250000	0	1	0.339623	0.178082	0	0	0.770992	0	0.225806	2	0	2
3	0.562500	1	1	0.245283	0.251142	0	1	0.816794	0	0.129032	2	0	2
4	0.583333	0	0	0.245283	0.520548	0	1	0.702290	1	0.096774	2	0	2

- Use normalization for distance-based models or when a specific range is required.